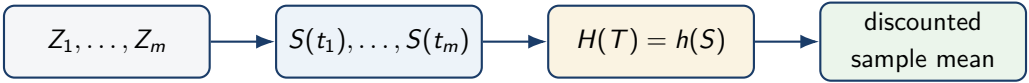
Monte Carlo and Path-Dependent Options

FRE6883 – Recitation 11

Simulate a path, evaluate a payoff, quantify the uncertainty

One path, one payoff, one sample



By the end

You can trace the simulation loop, implement an arithmetic Asian payoff, compute price and standard error, estimate delta with shared paths, and explain a control variate.

What makes an option path-dependent?

European call

$$H(T) = (S(T) - K)^+.$$

Only the terminal price matters.

Arithmetic Asian call

$$H(T) = \left(\frac{1}{m} \sum_{k=1}^{m} S(t_k) - K \right)^+.$$

Every monitored price contributes.

Diagnostic question

Can two paths finish at the same $S(T)$ but produce different Asian payoffs? Give an example.

The risk-neutral path update

For $\Delta t = T/m$ and independent $Z_k \sim N(0,1)$,

$$S(t_k) = S(t_{k-1}) \exp\left[\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\, Z_k\right].$$

Read the formula

The previous price is multiplied by a positive random growth factor. The same update repeats at each monitoring date with a new normal draw.

GUIDED CODE Numerical trace

Compute before revealing

Let $S_0 = 100$, $r = 0.03$, $\sigma = 0.20$, $\Delta t = 0.25$, and $Z_1 = 1$.

$$S_1 = 100 \exp\left[\left(0.03 - \frac{0.20^2}{2}\right) 0.25 + 0.20\sqrt{0.25}\right].$$

Which term is deterministic? Which term changes from one simulated path to another?

$$\left(r - \frac{\sigma^2}{2}\right)\Delta t = 0.0025, \qquad \sigma\sqrt{\Delta t}\, Z_1 = 0.10.$$

Hence

$$S_1 = 100e^{0.1025} \approx 110.79.$$

Interpretation

The drift contribution is fixed by the model inputs; the diffusion contribution changes with the normal draw.

GUIDED CODE Payoff trace

Average first, then apply the strike

For $K = 100$ and the simulated path

$$S = (98, , 102, , 105, , 99),$$

compute

$$H(T) = \left(\frac{1}{4} \sum_{k=1}^{4} S_k - K \right)^+ .$$

Would the payoff change if the path values were reordered?

02 | Evaluate an Asian payoff – Solution

$$\overline{S} = \frac{98 + 102 + 105 + 99}{4} = 101, \qquad H(T) = \max(101 - 100, 0) = 1.$$

Order versus collection

For this arithmetic-average payoff, reordering the same monitored values does not change the result. Other path-dependent contracts, such as barrier or lookback options, can depend on order or extrema.

Box–Muller turns uniforms into a normal draw

If U_1, U_2 are independent and uniform on $(0, 1]$, then

$$Z = \sqrt{-2 \log U_1} \cos(2\pi U_2)$$

has the standard normal distribution.

Why exclude zero?

$\log(0)$ is undefined. The lecture code uses `(rand()+1.0)/(RAND_MAX+1.0)` so the sampled value lies in $(0, 1]$.

03 | Complete `Gauss()` – Problem

GUIDED CODE Random input

Fill the transformation

```
const double pi = 4.0 * atan(1.0);

double Gauss()
{
    double U1 = (rand() + 1.0)
                / (RAND_MAX + 1.0);
    double U2 = (rand() + 1.0)
                / (RAND_MAX + 1.0);
    return /* Box--Muller expression */;
}
```

03 | Complete `Gauss()` – Solution

```
double Gauss()
{
    double U1 = (rand() + 1.0)
                / (RAND_MAX + 1.0);
    double U2 = (rand() + 1.0)
                / (RAND_MAX + 1.0);
    return sqrt(-2.0 * log(U1))
           * cos(2.0 * pi * U2);
}
```

Include `<cstdlib>` for `rand` and `<cmath>` for `sqrt`, `log`, `cos`, and `atan`.

The model owns market inputs; the caller owns the path

```cpp
typedef std::vector<double> SamplePath;

class MCModel {
private:
    double S0, r, sigma;
public:
    MCModel(double S0_, double r_, double sigma_)
      : S0(S0_), r(r_), sigma(sigma_) {
        srand((unsigned)time(NULL));
    }
    void GenerateSamplePath(double T, int m,
                            SamplePath& S) const;
    double GetS0() const { return S0; }
    double GetR() const { return r; }
};
```

GUIDED CODE Core simulation loop — Complete `dt` and the update

```
void MCModel::GenerateSamplePath(
    double T, int m, SamplePath& S) const
{
    double St = S0;
    double dt = /* ? */;
    for (int k = 0; k < m; ++k)
    {
        S[k] = St * exp(
            /* risk-neutral log return */);
        St = S[k];
    }
}
```

Precondition

The caller must create `S` with at least `m` elements.

04 | Generate the sample path – Solution

```
double dt = T / m;
for (int k = 0; k < m; ++k)
{
    S[k] = St * exp(
        (r - 0.5 * sigma * sigma) * dt
        + sigma * sqrt(dt) * Gauss());
    St = S[k];
}
```

State update

Each new point starts from the previously simulated price. Replacing `St` by `S0` would simulate independent one-step prices rather than one path.

05 | Find the path-storage bug

GUIDED CODE Indexing

What can go wrong?

```
SamplePath S;
model.GenerateSamplePath(T, m, S);
```

05 | Find the path-storage bug

GUIDED CODE Indexing

What can go wrong?

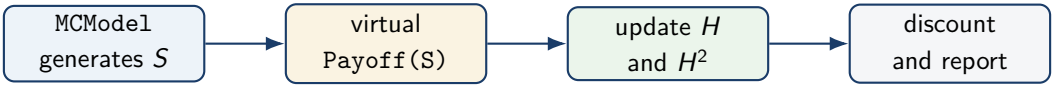
```
SamplePath S;
model.GenerateSamplePath(T, m, S);
```

Answer

`S` has size zero, but `GenerateSamplePath` writes `S[0]` through `S[m-1]`. Raw indexing does not grow a vector.

```
SamplePath S(m);      // allocate m elements first
```

Architecture of one Monte Carlo price



Separation of responsibilities

The model simulates the underlying. A derived option defines the payoff. The base option class owns the reusable Monte Carlo loop.

The base class defines the pricing protocol

```
class PathDepOption {
protected:
    double Price;
    double K, T;
    int m;
public:
    PathDepOption(double K_, double T_, int m_)
      : Price(0.0), K(K_), T(T_), m(m_) {}
    virtual ~PathDepOption() {}
    virtual double Payoff(
        const SamplePath& S) const = 0;
    double PriceByMC(const MCModel& Model, long N);
    double GetPrice() { return Price; }
};
```

Why pure virtual?

There is no generic path-dependent payoff. Every concrete option must supply one.

The arithmetic Asian call supplies the payoff

```
class ArthmAsianCall : public PathDepOption {
public:
    ArthmAsianCall(double K_, double T_, int m_)
      : PathDepOption(K_, T_, m_) {}

    double Payoff(
        const SamplePath& S) const;
};
```

Read the signature

`const SamplePath&` avoids copying the vector and promises not to modify it. The final `const` promises not to modify the option object.

06 | Complete the Asian payoff – Problem

GUIDED CODE Polymorphic payoff

Fill the running average and return value

```
double ArthmAsianCall::Payoff(
    const SamplePath& S) const
{
    double Ave = 0.0;
    for (int k = 0; k < m; ++k)
        Ave = /* update using S[k] */;

    if (Ave < K) return 0.0;
    return /* ? */;
}
```

06 | Complete the Asian payoff – Solution

```
double Ave = 0.0;
for (int k = 0; k < m; ++k)
    Ave = (k * Ave + S[k]) / (k + 1.0);

if (Ave < K) return 0.0;
return Ave - K;
```

Invariant

After iteration `k`, `Ave` is the average of `S[0]` through `S[k]`.

GUIDED CODE Numerical state Update without storing all payoffs

Suppose the first three payoffs are

$$h_1 = 0, qquad h_2 = 3, qquad h_3 = 6.$$

Starting from $H = 0$, apply

$$H \leftarrow \frac{iH + h_{i+1}}{i+1}$$

for $i = 0, 1, 2$.

Question

What does H represent after each iteration?

07 | Trace the online mean – Solution

$$H_1 = 0, qquad H_2 = \frac{0+3}{2} = 1.5, qquad H_3 = \frac{2(1.5)+6}{3} = 3.$$

Invariant

After $i+1$ samples, H equals the sample mean of all payoffs seen so far. Only the current mean and the new payoff are needed.

08 | Complete `PriceByMC` – Problem

GUIDED CODE Reusable pricing loop

Fill simulation, payoff, and discounting

```
double PathDepOption::PriceByMC(
    const MCModel& Model, long N)
{
    double H = 0.0;
    SamplePath S(m);
    for (long i = 0; i < N; ++i)
    {
        /* generate one path */;
        double payoff = /* virtual call */;
        H = /* online mean */;
    }
    Price = /* discount H */;
    return Price;
}
```

08 | Complete PriceByMC – Solution

```
double H = 0.0;
SamplePath S(m);
for (long i = 0; i < N; ++i)
{
    Model.GenerateSamplePath(T, m, S);
    double payoff = Payoff(S);
    H = (i * H + payoff) / (i + 1.0);
}
Price = exp(-Model.GetR() * T) * H;
return Price;
```

Polymorphism in the loop

`Payoff(S)` dispatches to the derived option. The simulation and averaging code stays in the base class.

GUIDED CODE Dynamic dispatch

Predict the call target

```
ArthmAsianCall option(100.0, 1.0 / 12.0, 30);
PathDepOption* ptr = &option;

double price = ptr->PriceByMC(model, 30000);
```

Inside `PriceByMC`, which implementation handles `Payoff(S)`?

GUIDED CODE Dynamic dispatch — Predict the call target

```
ArthmAsianCall option(100.0, 1.0 / 12.0, 30);
PathDepOption* ptr = &option;

double price = ptr->PriceByMC(model, 30000);
```

Inside `PriceByMC`, which implementation handles `Payoff(S)`?

Answer

`ArthmAsianCall::Payoff`. The base pointer retains the dynamic type of the object to which it points.

End-to-end driver from the lecture

```
double S0 = 100.0, r = 0.03, sigma = 0.2;
MCModel model(S0, r, sigma);

double T = 1.0 / 12.0, K = 100.0;
int m = 30;
ArthmAsianCall option(K, T, m);

long N = 30000;
option.PriceByMC(model, N);
std::cout << option.GetPrice() << std::endl;
```

Do not expect an identical last digit

The seed depends on calendar time, so repeated runs use different pseudo-random paths and produce different estimates.

A Monte Carlo price is an estimate

Let H_i be payoff sample i. The discounted estimator is

$$\widehat{V}_N = e^{-rT} \frac{1}{N} \sum_{i=1}^{N} H_i.$$

Its estimated standard error is

$$\widehat{\mathrm{SE}}_N = e^{-rT} \sqrt{\frac{\overline{H^2} - \overline{H}^2}{N-1}}.$$

Meaning

The standard error measures sampling uncertainty in the estimated price. It is not the option payoff's standard deviation.

GUIDED CODE Error scaling

Vote before calculating

Monte Carlo standard error scales approximately as

$$\mathrm{SE}_N \propto \frac{1}{\sqrt{N}}.$$

To divide the standard error by 10, should we use:

1. $10N$ paths,
2. $20N$ paths,
3. $100N$ paths?

$$\frac{\mathrm{SE}_{cN}}{\mathrm{SE}_N} = \frac{1/\sqrt{cN}}{1/\sqrt{N}} = \frac{1}{\sqrt{c}}.$$

Setting $1/\sqrt{c} = 1/10$ gives $c = 100$.

Computational consequence

One additional decimal digit of Monte Carlo accuracy can cost about one hundred times more paths. Variance reduction attacks this cost directly.

Track the first two sample moments

For each payoff h, update

$$H \leftarrow \frac{iH + h}{i+1}, \qquad H_{\mathrm{sq}} \leftarrow \frac{iH_{\mathrm{sq}} + h^2}{i+1}.$$

Then H estimates $E[H(T)]$ and H_{sq} estimates $E[H(T)^2]$.

Variance identity

$$\mathrm{Var}(H(T)) = E[H(T)^2] - E[H(T)]^2.$$

11 | Add the pricing error – Problem

GUIDED CODE Two moments

Complete the loop and final formula

```
double H = 0.0, Hsq = 0.0;
for (long i = 0; i < N; ++i)
{
    Model.GenerateSamplePath(T, m, S);
    double payoff = Payoff(S);
    H = (i * H + payoff) / (i + 1.0);
    Hsq = /* update mean square */;
}
Price = exp(-Model.GetR() * T) * H;
PricingError = /* standard error */;
```

11 | Add the pricing error – Solution

```
Hsq = (i * Hsq + payoff * payoff) / (i + 1.0);

Price = exp(-Model.GetR() * T) * H;
PricingError = exp(-Model.GetR() * T)
    * sqrt(Hsq - H * H) / sqrt(N - 1.0);
```

Class change

Add `PricingError` as a data member and expose `GetPricingError()` after `PriceByMC` has populated both outputs.

12 | Find the repeated-work bug

GUIDED CODE Efficiency and consistency

What should be changed?

```
H = (i * H + Payoff(S)) / (i + 1.0);
Hsq = (i * Hsq + pow(Payoff(S), 2.0))
      / (i + 1.0);
```

12 | Find the repeated-work bug

GUIDED CODE Efficiency and consistency

What should be changed?

```
H = (i * H + Payoff(S)) / (i + 1.0);
Hsq = (i * Hsq + pow(Payoff(S), 2.0))
      / (i + 1.0);
```

Better

Evaluate once and reuse the result.

```
double payoff = Payoff(S);
H = (i * H + payoff) / (i + 1.0);
Hsq = (i * Hsq + payoff * payoff) / (i + 1.0);
```

Delta by a finite difference

Let $u(S_0)$ be the option price. For small ε,

$$\Delta = \frac{du}{dS_0}(S_0) \approx \frac{u((1+\varepsilon)S_0) - u(S_0)}{\varepsilon S_0}.$$

Under geometric Brownian motion, multiplying S_0 by $1+\varepsilon$ multiplies every point on a path by the same factor.

Implementation idea

Generate one base path S, evaluate its payoff, rescale the same path, and evaluate the bumped payoff.

Why reuse the same path?

Independent paths

The base and bumped estimates contain unrelated simulation noise. Their difference can be noisy.

Common random numbers

The same $Z_1, \ldots, Z_m$ drive both payoffs. Much of the noise cancels in their difference.

Question

Does path reuse remove the finite-difference approximation error? Why not?

13 | Complete `Rescale` – Problem

GUIDED CODE Finite-difference helper

Modify every point in place

```
void Rescale(SamplePath& S, double x)
{
    int m = S.size();
    for (int j = 0; j < m; ++j)
        /* ? */;
}
```

What does `Rescale(S, 1.0 + epsilon)` represent financially?

13 | Complete `Rescale` – Solution

```
void Rescale(SamplePath& S, double x)
{
    int m = S.size();
    for (int j = 0; j < m; ++j)
        S[j] = x * S[j];
}
```

Financial meaning

`x = 1.0 + epsilon` produces the path associated with a bumped initial spot under the same sequence of random shocks.

14 | Add delta to the Monte Carlo loop – Problem

GUIDED CODE Shared paths

Complete the bumped accumulation

```
double H = 0.0, Hsq = 0.0, Heps = 0.0;
for (long i = 0; i < N; ++i)
{
    Model.GenerateSamplePath(T, m, S);
    double payoff = Payoff(S);
    H = (i * H + payoff) / (i + 1.0);
    Hsq = (i * Hsq + payoff * payoff) / (i + 1.0);

    /* bump the same path */;
    Heps = /* online mean of bumped payoff */;
}
delta = /* discounted finite difference */;
```

14 | Add delta to the Monte Carlo loop – Solution

```
Rescale(S, 1.0 + epsilon);
double payoffEps = Payoff(S);
Heps = (i * Heps + payoffEps) / (i + 1.0);

delta = exp(-Model.GetR() * T)
    * (Heps - H) / (Model.GetS0() * epsilon);
```

Order matters

Compute the base payoff before mutating the path. The rescaled vector is the bumped scenario.

Variance reduction changes the estimator

Suppose X is the target payoff and Y is a similar payoff with known price

$$V_Y = e^{-rT} E[Y].$$

Then

$$V_X = e^{-rT} E[X - Y] + V_Y.$$

Control variate idea

If X and Y move together, the difference $X - Y$ is less variable than X. Simulate the difference and add back the known value.

15 | Choose a useful control – Problem

GUIDED CODE Financial design

Which candidate is best?

Target: arithmetic Asian call payoff.

1. A digital option on an unrelated stock.
2. A geometric Asian call on the same monitored path.
3. A deterministic cash payment of one dollar.

Two requirements

The control should be strongly related to the target, and its time-zero value should be available without the same simulation problem.

The geometric Asian call is the natural control:

$$Y = \left(\left(\prod_{k=1}^{m} S(t_k) \right)^{1/m} - K \right)^{+}.$$

Why it works

Arithmetic and geometric averages are computed from the same path and usually move closely together. The geometric Asian call also has a closed-form price under the model used in the lecture.

16 | Build the difference option – Problem

GUIDED CODE Composition with pointers

Complete the payoff

```
class DifferenceOfOptions : public PathDepOption {
private:
    PathDepOption* Ptr1;
    PathDepOption* Ptr2;
public:
    DifferenceOfOptions(double K_, double T_, int m_,
                        PathDepOption* p1,
                        PathDepOption* p2)
      : PathDepOption(K_, T_, m_), Ptr1(p1), Ptr2(p2) {}

    double Payoff(const SamplePath& S) const {
        return /* ? */;
    }
};
```

16 | Build the difference option – Solution

```
double Payoff(const SamplePath& S) const
{
    return Ptr1->Payoff(S) - Ptr2->Payoff(S);
}
```

Dynamic dispatch twice

Each pointer invokes the payoff belonging to its own dynamic option type. The wrapper therefore represents $X - Y$ without copying either option.

The control option adds one pricing capability

```
class GmtrAsianCall : public PathDepOption {
public:
    GmtrAsianCall(double K_, double T_, int m_)
      : PathDepOption(K_, T_, m_) {}

    double Payoff(const SamplePath& S) const;
    double PriceByBSFormula(
        const MCModel& Model);
};
```

Two roles

`Payoff` lets the difference option use the control path by path. `PriceByBSFormula` supplies the known V_Y added after simulation.

17 | Complete variance-reduced pricing – Problem

GUIDED CODE Reuse the existing pricer

Fill two expressions

```
double PathDepOption::PriceByVarRedMC(
    const MCModel& Model, long N,
    PathDepOption& CVOption, double epsilon)
{
    DifferenceOfOptions VarRedOpt(
        K, T, m, /* target */, /* control */);

    Price = VarRedOpt.PriceByMC(
                Model, N, epsilon)
            + /* known control price */;
    PricingError = VarRedOpt.PricingError;
    return Price;
}
```

17 | Complete variance-reduced pricing – Solution

```
DifferenceOfOptions VarRedOpt(
    K, T, m, this, &CVOption);

Price = VarRedOpt.PriceByMC(Model, N, epsilon)
        + CVOption.PriceByBSFormula(Model);
PricingError = VarRedOpt.PricingError;
return Price;
```

Read `this`

`this` points to the target option on which `PriceByVarRedMC` was called. `&CVOption` points to the supplied control object.

Direct MC versus control variate

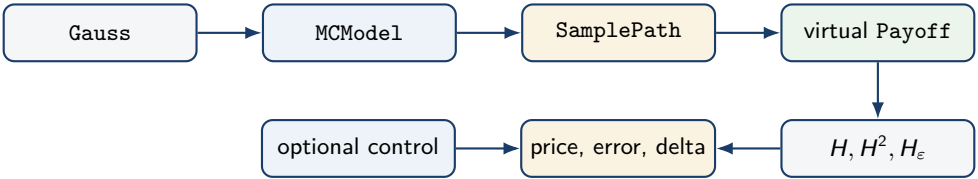
Illustrative lecture output for the arithmetic Asian call:

Estimator	Price	Standard error
Direct Monte Carlo	about 1.42	about 0.012
With geometric control	about 1.42	much smaller

What should remain stable?

Both estimators target the same price. Variance reduction should primarily reduce the reported sampling error, not systematically change the economic value.

End-to-end dependency map



Debug in this order

Validate random draws, then one path, then one payoff, then the sample moments, and only then the final discounted outputs.

Exit ticket

Explain

Why does `SamplePath S(m)` appear outside the simulation loop, while its values are replaced inside the loop?

Predict

If N is multiplied by 4, what happens approximately to the standard error?

Connect

Why does the finite-difference delta reuse one path for base and bumped payoffs?

Choose

What makes the geometric Asian call a useful control variate?

Reserve exercises

Optional challenges and additional diagnostics

R1 | Gamma by finite differences – Problem

GUIDED CODE Challenge — Design the three-scenario estimate

Using

$$\Gamma \approx \frac{u((1+\varepsilon)S_0) - 2u(S_0) + u((1-\varepsilon)S_0)}{(\varepsilon S_0)^2},$$

describe how one base path can produce the three payoffs needed in the numerator.

Constraint

All three scenarios should use the same underlying normal draws.

R1 | Gamma by finite differences – Solution

```
SamplePath base = S;
SamplePath up = base;
SamplePath down = base;
Rescale(up, 1.0 + epsilon);
Rescale(down, 1.0 - epsilon);

double h0 = Payoff(base);
double hUp = Payoff(up);
double hDown = Payoff(down);
```

Accumulate the three sample means and apply the discounted central-difference formula after the loop.

Tradeoff

The symmetric formula reduces finite-difference bias, while shared paths reduce simulation noise.

R2 | Geometric average – Find the risk

GUIDED CODE Challenge

Think about numerical range

```
double product = 1.0;
for (int k = 0; k < m; ++k)
    product *= S[k];
double geom = pow(product, 1.0 / m);
```

For long paths or large prices, what numerical failure could occur before the mth root is taken?

GUIDED CODE Challenge

Think about numerical range

```
double product = 1.0;
for (int k = 0; k < m; ++k)
    product *= S[k];
double geom = pow(product, 1.0 / m);
```

For long paths or large prices, what numerical failure could occur before the mth root is taken?

Answer

The product can overflow even when the final geometric mean is moderate.

R2 | Geometric average in log space

```
double logSum = 0.0;
for (int k = 0; k < m; ++k)
    logSum += log(S[k]);

double geom = exp(logSum / m);
return std::max(geom - K, 0.0);
```

Why this is safer

Products become sums of logarithms, which greatly extends the usable numerical range. Simulated GBM prices are positive, so their logarithms are defined.

R3 | Random seed as a testing decision

Time-based seed

Different runs produce different paths. Fresh paths across runs; harder to debug.

Fixed seed

The same seed reproduces the same pseudo-random sequence. Repeatable paths; useful for tests.

Question

Why should seeding happen once rather than every time `Gauss()` is called?

R4 | What does `this` point to?

GUIDED CODE Challenge — Identify type and address

```
ArthmAsianCall option(K, T, m);
GmtrAsianCall control(K, T, m);

option.PriceByVarRedMC(
    model, N, control, epsilon);
```

Inside `PriceByVarRedMC`, `this` has type `PathDepOption*`, but which object's address does it hold?

R4 | What does `this` point to?

GUIDED CODE Challenge — Identify type and address

```
ArthmAsianCall option(K, T, m);
GmtrAsianCall control(K, T, m);

option.PriceByVarRedMC(
    model, N, control, epsilon);
```

Inside `PriceByVarRedMC`, `this` has type `PathDepOption*`, but which object's address does it hold?

Answer

It holds `&option`, the address of the object that invoked the member function. The object's dynamic type remains `ArthmAsianCall`.

R5 | Turn standard error into an interval

For a large independent sample, an approximate 95% interval is

$$\widehat{V}_N \pm 1.96\,\widehat{\mathrm{SE}}_N.$$

Problem

If the price estimate is 1.420 and the standard error is 0.012, compute the interval.

R5 | Turn standard error into an interval

For a large independent sample, an approximate 95% interval is

$$\widehat{V}_N \pm 1.96\,\widehat{\mathrm{SE}}_N.$$

Problem

If the price estimate is 1.420 and the standard error is 0.012, compute the interval.

$$1.420 \pm 1.96(0.012) = 1.420 \pm 0.0235,$$

so the interval is approximately $[1.3965, 1.4435]$.

R6 | A deterministic diagnostic case

Set $\sigma = 0$. Then the simulated path is deterministic:

$$S(t_k) = S_0 e^{rt_k}.$$

Why this is a strong test

Every Monte Carlo path should be identical, the reported standard error should be zero up to roundoff, and the price should equal the discounted payoff computed from that deterministic path.

Bugs this can expose

Wrong time step, missing state update, incorrect averaging, incorrect discounting, or an uninitialized path element.

Monte Carlo – One-page cheat sheet

One sample

Generate normals → build path → evaluate virtual payoff → update running moments.

Outputs

Price $= e^{-rT}\overline{H}$. Standard error uses $\overline{H^2} - \overline{H}^2$ and falls like $N^{-1/2}$.

Greeks

Rescale the same path for the bumped spot. Shared randomness reduces noise in the finite difference.

Control variate

Simulate $X - Y$, then add the known price of Y. Choose a control that moves closely with the target.